

09

VEJA PARA CRER: DASHBOARDS E INSIGHTS VISUAIS COM IOT E IA

PDF exclusivo para Murilo Salla - rm568041
murilosalla@protonmail.com

LISTA DE FIGURAS

Figura 1 - Esquema simplificado de um sistema ciberfísico com dashboard de monitoramento 7
Figura 2 - Exemplo de dashboard com boas práticas de design para sistemas IoT .10
Figura 3 - Fluxograma do Node-RED mostrando todas as etapas de processamento 14
Figura 4 - Dashboard Node-RED com leitura em tempo real da luminosidade.....17
Figura 5 - Fluxo Node-RED com recepção MQTT em tempo real.....19
Figura 6 - Conexão física entre o sensor de luminosidade GY-30 (BH1750) e o microcontrolador ESP32 utilizando a interface I²C.....20
Figura 7 - Fluxo de envio de alerta de luminosidade com integração ao Telegram via Node-RED.....24
Figura 8 - Dashboard com gauge de luminosidade e alerta visual para valores acima de 400 lux.....26
Figura 9 - Alerta enviado via Telegram informando o excesso de luminosidade detectado pelo sensor.....27
Figura 10 - Estrutura lógica do InfluxDB para dados de sensores IoT 31
Figura 11 - Dashboard do Grafana com gráficos e filtros temporais 32
Figura 12 - Arquitetura de integração: ESP32, MQTT, InfluxDB, Node-RED e Grafana 34
Figura 13 - Painel no Grafana com leitura histórica de luminosidade (BH1750) conectada ao InfluxDB 35
Figura 14 - Exemplo de dashboard no ThingsBoard com indicadores de consumo, corrente, potência, alarmes e contato 36
Figura 15 - Arquitetura conceitual do ThingsBoard, destacando fluxos entre dispositivos, protocolos, engine de regras e bases de dados 37
Figura 16 - Editor gráfico do Rule Engine, com um fluxo condicional para tratamento de atributos e telemetria.....38

FIAP

LISTA DE QUADROS

Quadro 1 - Classificação de variáveis comumente monitoradas em projetos de IoT 8
Quadro 2 - Comparativo entre plataformas de dashboard para IoT 41

FIAP

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 - Comandos para instalação do Node.js, Node-RED e Mosquitto no Windows....13
Código-fonte 2 - Configuração do Mosquitto e inicialização dos serviços....13
Código-fonte 3 - Instalação do painel de dashboards no Node-RED....14
Código-fonte 4 - Código de envio de dados do ESP32 para o tópico MQTT....18
Código-fonte 5 - Envio de dados do sensor BH1750 via MQTT utilizando ESP32....22
Código-fonte 6 - Código de detecção e envio de alerta de luminosidade via Telegram no Node-RED....24
Código-fonte 7 - Publicação de dados do sensor GY-91 em tópicos distintos via MQTT....33

FIAP

SUMÁRIO

1 VEJA PARA CRER: DASHBOARDS E INSIGHTS VISUAIS COM IOT E IA 6
1.1 Conceito de dashboard em sistemas ciberfísicos 6
1.2 Requisitos para dashboards em IoT: Tempo real, histórico e alertas.....7
1.3 Métricas e variáveis comuns a serem monitoradas.....8
1.4 Boas práticas de design de dashboards para IoT 8

2 NODE-RED COMO PLATAFORMA DE VISUALIZAÇÃO 12
2.1 O que é o Node-RED e como funciona? 12
2.2 Instalação local do Node-RED com Mosquitto 13
2.3 Construção de dashboards com gauges, gráficos e indicadores 14
2.4 Criação de tópicos MQTT e leitura em tempo real 18
2.5 Leitura de luminosidade com GY-30 19
2.6 Alertas inteligentes com Node-RED 22
2.6.1 Condições para disparo de alerta.....23
2.6.2 Envio de alertas via Telegram 23
2.6.3 Alerta de luminosidade via Telegram 25
2.7 Análise dos resultados 28

3 GRAFANA + INFLUXDB PARA ANÁLISE HISTÓRICA 30
3.1 Introdução ao InfluxDB como banco de séries temporais 30
3.2 Introdução ao Grafana como ferramenta de visualização 31
3.3 Integração de dados com InfluxDB e visualização em painéis Grafana.....32

4 THINGSBOARD COMMUNITY EDITION.....36

5 COMPARAÇÃO ENTRE PLATAFORMAS.....39
5.1 Interface visual e curva de aprendizado.....39
5.2 Integração, armazenamento e visualização 39
5.3 Funcionalidades adicionais, robustez e escalabilidade 40
5.4 Quadro comparativo.....41

REFERÊNCIAS.....42

FIAP

1 VEJA PARA CRER: DASHBOARDS E INSIGHTS VISUAIS COM IOT E IA

A coleta de dados em sistemas de Internet das Coisas (IoT) é apenas o primeiro passo para que se possa gerar informação significativa. A etapa seguinte é a visualização dos dados, o que permite análises, tomadas de decisão e respostas automatizadas. Neste contexto, os dashboards se tornaram ferramentas essenciais, oferecendo — geralmente em tempo real — interfaces gráficas que representam o estado atual e o histórico das variáveis monitoradas.

Dashboards em IoT devem ser construídos com foco em responsividade, clareza, atualização em tempo real (quando necessário) e capacidade de adaptação para diferentes tipos de dispositivos. Neste capítulo, exploraremos diversas abordagens de construção de dashboards, integrando sensores embarcados em ESP32 com plataformas de visualização como Node-RED, Grafana e ThingsBoard.

O capítulo está estruturado em cinco seções. Inicialmente, são apresentados os fundamentos de dashboards em IoT, seguidos de três plataformas distintas com experiências práticas. Em seguida, discute-se o uso de alertas inteligentes e a comparação entre as plataformas utilizadas. Por fim, são apresentadas as conclusões e aplicações reais.

1.1 Conceito de dashboard em sistemas ciberfísicos

Em sistemas ciberfísicos, os dashboards representam a interface entre o mundo físico e o mundo digital. Esses sistemas envolvem sensores, atuadores e unidades computacionais conectadas em rede, e os dashboards são responsáveis por condensar os dados capturados em tempo real e apresentar insights relevantes de forma clara e interativa. Além disso, funcionam como pontos de controle, nos quais decisões automáticas ou humanas podem ser tomadas com base em leituras atuais ou em tendências históricas.

A figura “Esquema simplificado de um sistema ciberfísico com dashboard de monitoramento” apresenta um exemplo simplificado para ilustrar os conceitos fundamentais de um sistema ciberfísico, destacando a integração entre o mundo físico, o processamento computacional e a visualização de dados. Nesse exemplo,

um sensor de temperatura coleta informações do ambiente e as envia para um ESP32, que atua como unidade de processamento local. Com base nos dados recebidos, o ESP32 pode acionar um atuador, representando a capacidade de resposta física do sistema. Paralelamente, os dados são transmitidos via Wi-Fi para um broker MQTT, que gerencia a comunicação entre os dispositivos e a nuvem. A partir desse ponto, as informações são disponibilizadas em um dashboard, que exibe gráficos, indicadores e alertas em tempo real.

Essa estrutura exemplifica como os sensores, os atuadores, a conectividade e as interfaces de visualização se articulam em um sistema ciberfísico, permitindo supervisão contínua e tomada de decisão baseada em dados.

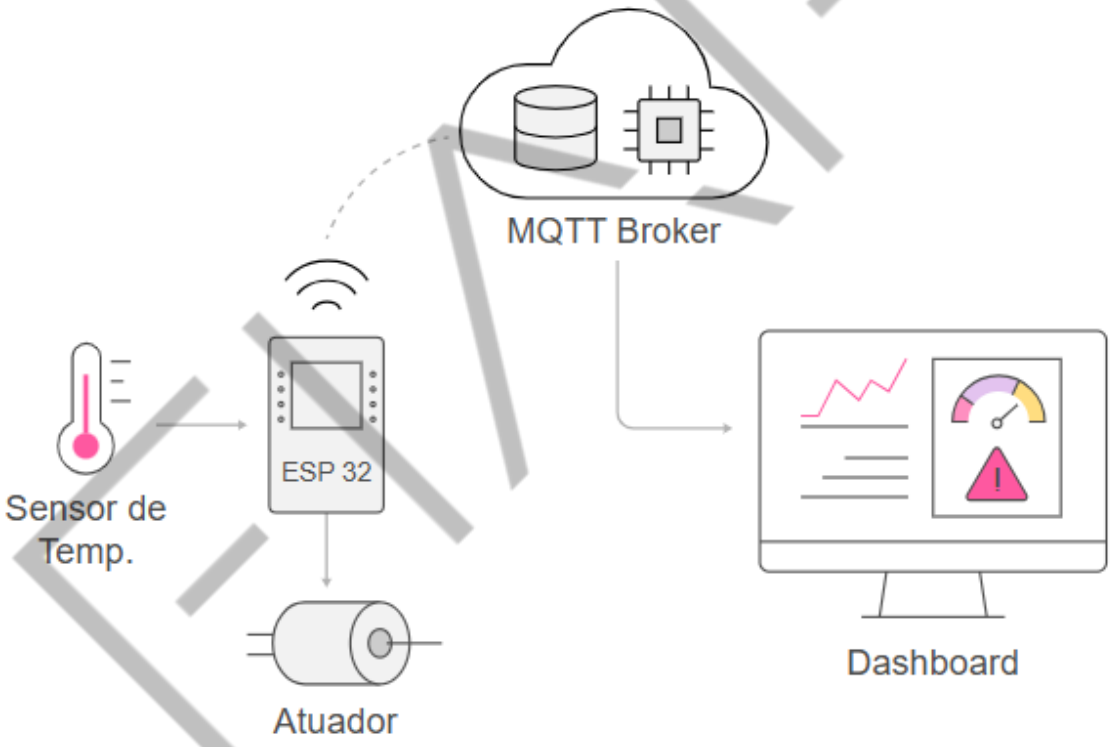


Figura 1 - Esquema simplificado de um sistema ciberfísico com dashboard de monitoramento
Fonte: Elaborado pelo autor (2025)

1.2 Requisitos para dashboards em IoT: Tempo real, histórico e alertas

Para que um dashboard seja eficaz em aplicações IoT, ele deve contemplar alguns requisitos fundamentais. A atualização em tempo real é necessária em aplicações críticas, como no monitoramento de ambientes industriais ou na saúde, permitindo respostas imediatas. O armazenamento de dados históricos, por sua vez, é essencial para análise de tendências, comparação de dados entre períodos distintos

e auditorias. Já a funcionalidade de alertas e as notificações garante que o usuário seja informado de forma proativa sempre que um valor crítico for atingido.

Esses três pilares de atualização imediata, histórico consolidado e notificações reativas devem ser cuidadosamente implementados no projeto do dashboard, levando em consideração a frequência de amostragem, a capacidade de armazenamento e a criticidade das variáveis monitoradas.

1.3 Métricas e variáveis comuns a serem monitoradas

As variáveis mais comuns em projetos de IoT englobam grandezas ambientais como temperatura, umidade, luminosidade e qualidade do ar, geralmente obtidas por sensores como DHT22, GY-30 e MQ135. Também são recorrentes os parâmetros físicos, como aceleração, pressão e corrente elétrica, que podem ser monitorados por sensores como MPU6050, BMP280 e ACS712.

Além disso, em aplicações industriais e urbanas, indicadores de desempenho do sistema — como consumo energético, tempo de atividade de equipamentos, taxa de falhas ou eventos anômalos — são essenciais. Essas métricas permitem mensurar a eficiência do sistema e auxiliam na tomada de decisão automatizada ou humana.

O quadro a seguir apresenta uma classificação didática das variáveis comumente monitoradas em IoT.

Categoria	Variáveis comuns	Exemplo de sensor
Ambientais	Temperatura, umidade, luminosidade, qualidade do ar.	DHT22, GY-30, MQ135.
Parâmetros físicos	Aceleração, pressão, corrente elétrica, vibração.	MPU6050, BMP280, ACS712.
Indicadores de sistema	Consumo de energia, uptime, falhas, uso da rede.	Medido via software ou hardware dedicado.

Quadro 1 - Classificação de variáveis comumente monitoradas em projetos de IoT
Fonte: Elaborado pelo autor (2025)

1.4 Boas práticas de design de dashboards para IoT

O design de dashboards deve garantir clareza, eficiência e adaptabilidade. É fundamental que os dados sejam organizados de maneira hierárquica, com elementos

visuais destacados para os valores mais importantes. A escolha das cores, o uso de gauges, gráficos de linha, tabelas e alertas visuais são recursos que contribuem para a rápida interpretação das informações. Além disso, o layout deve ser responsivo, funcionando adequadamente em diferentes dispositivos e resoluções.

A figura “Exemplo de dashboard com boas práticas de design para sistemas IoT” apresenta um dashboard interativo construído com a ferramenta Node-RED, demonstrando boas práticas de design e organização de dados em aplicações IoT. Os dados são coletados em tempo real a partir de sensores de temperatura, umidade relativa do ar e concentração de CO_2 conectados ao sistema, simulando o funcionamento de uma aplicação embarcada em um ambiente físico.

O painel exibe três medidores circulares (gauges), que fornecem uma leitura instantânea dos valores capturados pelos sensores, e três gráficos de linha que mostram a evolução dessas variáveis ao longo do tempo. A interface foi projetada com foco em clareza e eficiência, utilizando interpolação linear, cores que variam conforme faixas de normalidade e limitação no número de pontos exibidos, facilitando a interpretação dos dados em tempo real.

Esse exemplo ilustra o potencial do Node-RED como plataforma visual de desenvolvimento para soluções IoT, permitindo integrar sensores, processar dados e apresentar visualizações dinâmicas de forma rápida e eficiente. A ferramenta se destaca por sua capacidade de representar sistemas ciberfísicos com coleta, análise e resposta visual em um único ambiente acessível e modular.

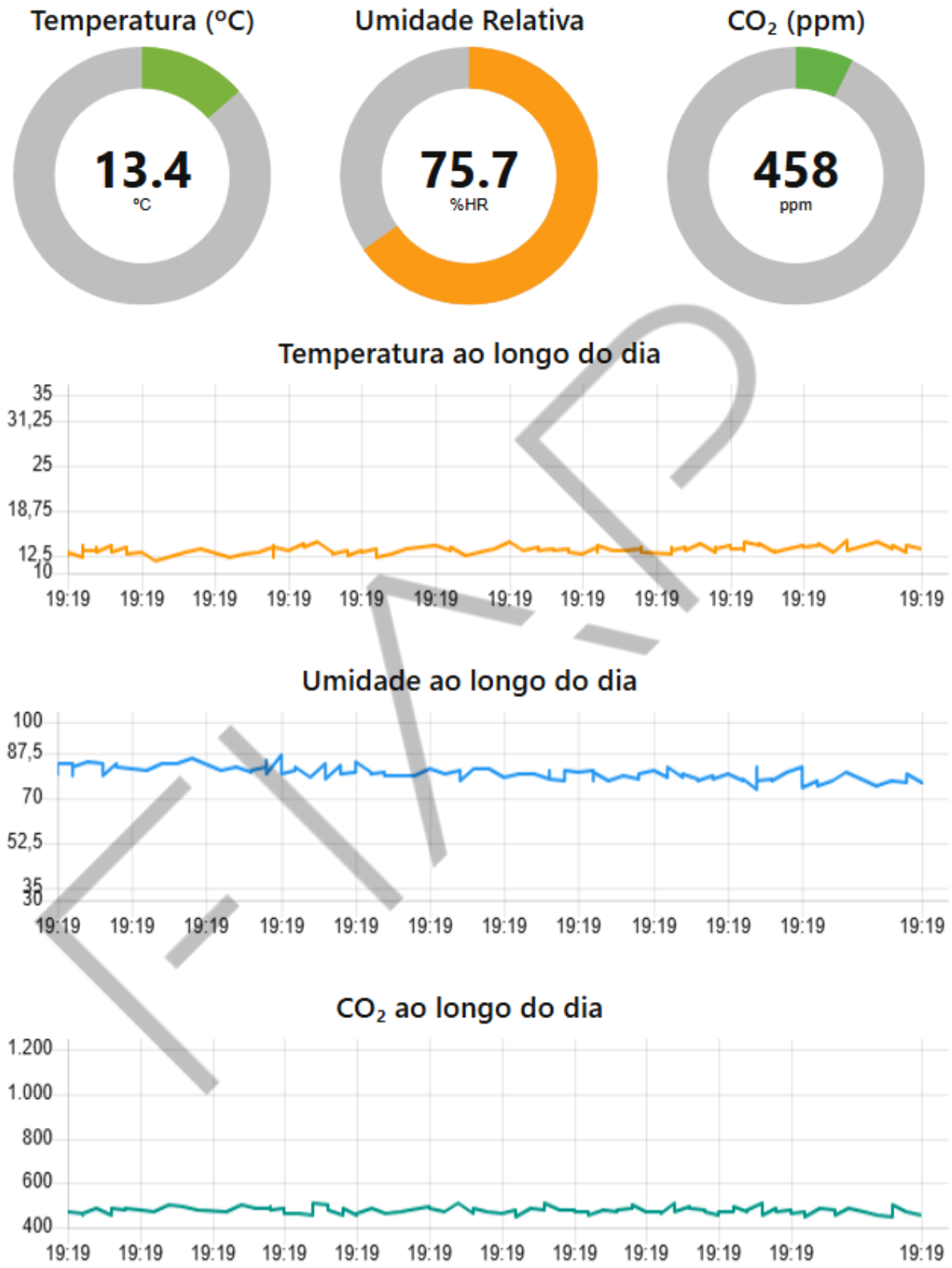


Figura 2 - Exemplo de dashboard com boas práticas de design para sistemas IoT
Fonte: Node-RED (2025)

Outro ponto importante é a modularidade, que permite a reutilização ou reorganização dos blocos de informação conforme a necessidade do usuário. Com isso, o dashboard se torna escalável e mais fácil de manter.

Com os fundamentos bem estabelecidos, a próxima seção apresenta o Node-RED, uma das ferramentas mais utilizadas para criação de dashboards em sistemas IoT. Nas seções seguintes, serão abordados seu funcionamento, instalação e as possibilidades de visualização em tempo real através de experiências práticas.

FIAP

2 NODE-RED COMO PLATAFORMA DE VISUALIZAÇÃO

O Node-RED é uma ferramenta de programação visual voltada para a integração de dispositivos físicos, APIs e serviços online. Sua interface intuitiva baseada em fluxos permite a criação de aplicações complexas com poucos cliques, utilizando blocos de funcionalidades (nós) que representam entradas, processamentos e saídas de dados. Essa abordagem facilita a prototipagem rápida de soluções em IoT, especialmente para monitoramento e visualização de dados em tempo real.

Para aplicações de IoT, o Node-RED é amplamente utilizado como ponte entre sensores conectados (por exemplo, via MQTT) e sistemas de armazenamento ou dashboards visuais. Com sua extensa biblioteca de nós, é possível construir fluxos que recebem dados de sensores embarcados no ESP32, realizam transformações, aplicam lógica de negócios e enviam os resultados para diferentes destinos, como bancos de dados, notificações, interfaces web e plataformas de visualização.

Neste capítulo, abordaremos desde a instalação local do Node-RED até sua integração com o broker MQTT e a criação de dashboards com elementos visuais como gráficos, indicadores e LEDs virtuais. Também será apresentada uma experiência prática com o sensor GY-30, demonstrando o processo completo de coleta, transmissão, recepção e visualização de dados de luminosidade.

A seguir, serão apresentados os conceitos fundamentais do Node-RED, sua arquitetura e os principais recursos disponíveis para visualização de dados em tempo real em aplicações IoT.

2.1 O que é o Node-RED e como funciona?

O Node-RED é uma ferramenta de desenvolvimento baseada em fluxo, criado originalmente pela IBM, que permite a integração de dispositivos, APIs e serviços online por meio de uma interface visual simples. Ao utilizar blocos conectados graficamente, o Node-RED permite a construção de lógicas de processamento e visualização de dados, sendo amplamente utilizado em projetos de IoT devido à sua facilidade e robustez.

Seu funcionamento baseia-se em três componentes principais:

- **Nós (nodes):** são blocos que representam funções específicas, como entrada de dados, processamento, saída, conexão com banco de dados ou interface gráfica.
- **Fluxos (flows):** conjunto de nós interligados que definem a lógica do sistema.
- **Dashboard:** interface de visualização criada a partir de nós gráficos, como medidores, gráficos, tabelas e botões.

Node-RED é especialmente poderoso quando integrado com o protocolo MQTT, permitindo a comunicação em tempo real entre microcontroladores como o ESP32 e a interface do usuário.

2.2 Instalação local do Node-RED com Mosquitto

Para instalar o Node-RED localmente em um ambiente de desenvolvimento Windows, são necessários os seguintes passos:

- **Passo 1:** instalar o Node.js (versão LTS recomendada).
- **Passo 2:** instalar o Node-RED via npm:

```
npm install -g --unsafe-perm node-red
```

Código-fonte 1 - Comandos para instalação do Node.js, Node-RED e Mosquitto no Windows
Fonte: Elaborado pelo autor (2025)

- **Passo 3:** instalar o broker Mosquitto:
  - Baixar o Mosquitto para Windows;
  - Configurar o arquivo mosquitto.conf para permitir conexões na porta 1883.
- **Passo 4:** iniciar o Mosquitto e o Node-RED:

```
mosquitto -v
node-red
```

Código-fonte 2 - Configuração do Mosquitto e inicialização dos serviços
Fonte: Elaborado pelo autor (2025)

Após o início do Node-RED, é possível acessá-lo via navegador em: http://localhost:1880. Para ativar a funcionalidade de dashboards, é necessário instalar o pacote adicional:

```
cd ~/.node-red
npm install node-red-dashboard
```

Código-fonte 3 - Instalação do painel de dashboards no Node-RED
Fonte: Elaborado pelo autor (2025)

2.3 Construção de dashboards com gauges, gráficos e indicadores

A construção de dashboards no Node-RED é uma etapa essencial para a apresentação clara e interativa dos dados coletados por dispositivos IoT. Ao utilizar blocos visuais de fácil configuração, é possível transformar dados brutos recebidos por sensores em painéis ricos, com indicadores gráficos, alertas em tempo real e notificações automáticas.

A interface principal do Node-RED pode ser acessada por meio do endereço http://localhost:1880/. Nessa interface, o usuário visualiza e constrói os fluxos utilizando os nós disponíveis na paleta lateral. Nesta subseção, será detalhado um experimento completo que realiza a leitura da luminosidade de um ambiente por meio de um sensor BH1750 conectado a um microcontrolador ESP32. Os dados são transmitidos via MQTT para o Node-RED, onde são processados, visualizados e utilizados no acionamento de alertas e indicadores.

A seguir, a figura mostra o fluxograma completo implementado no Node-RED. O fluxo se inicia com o nó “Recebe do ESP32”, um nó MQTT configurado para escutar o tópico ao qual o ESP32 publica os dados de luminosidade. Este microcontrolador está programado para realizar leituras periódicas com o sensor BH1750 e transmitir os valores em formato JSON, por exemplo: { "lux": 248.33 }.

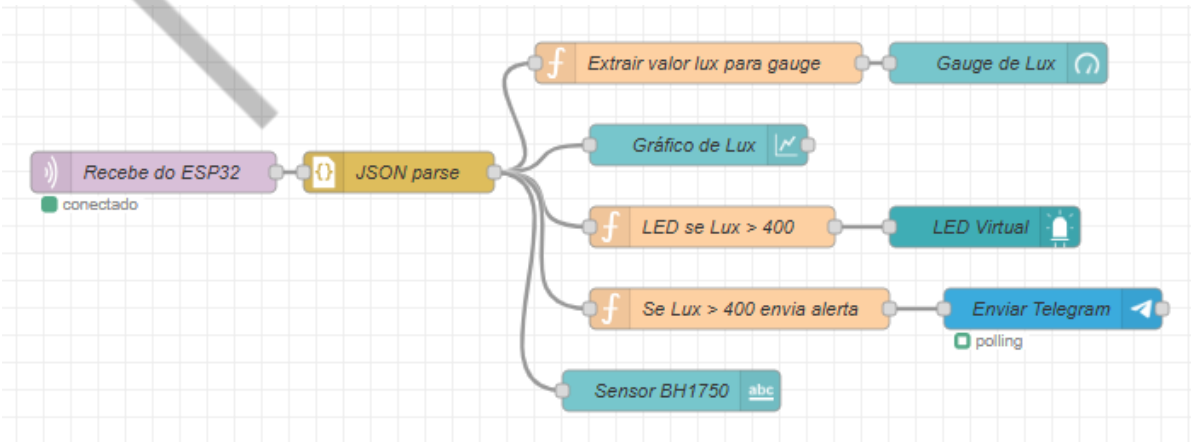


Figura 3 - Fluxograma do Node-RED mostrando todas as etapas de processamento
Fonte: Node-RED (2025)

O nó seguinte é o “JSON parse”, responsável por converter a string JSON em um objeto manipulável dentro do Node-RED. A partir dele, o fluxo se ramifica em cinco saídas distintas, cada uma desempenhando uma função específica no sistema:

Extração e formatação do valor para o gauge:

A primeira saída leva os dados a uma função nomeada “Extrair valor lux para gauge”. Este nó utiliza um código JavaScript simples para extrair apenas o valor numérico de lux, formatá-lo com duas casas decimais e enviá-lo ao gauge. Isso garante que o medidor circular apresente o valor de forma limpa e precisa.

Gráfico de linha (histórico de luminosidade):

A segunda saída leva diretamente o dado para o nó “Gráfico de Lux”, que utiliza o componente ui_chart para representar os dados ao longo do tempo. O gráfico está configurado para exibir apenas os últimos 30 registros — limitando visualmente o eixo X —, utiliza interpolação linear entre os pontos e exibe os horários formatados como HH:mm. Essa representação permite ao usuário analisar a tendência da luminosidade ao longo do tempo, detectando padrões como quedas bruscas ou aumentos graduais.

LED virtual para alerta visual no painel:

A terceira saída passa pelo nó “LED se Lux > 400”, que avalia se o valor da luminosidade está acima de 400 lux. Caso a condição seja verdadeira, é enviada uma mensagem para o LED virtual (nó ui_led) que acende automaticamente. Esse LED pode ser configurado para acender em verde (nível aceitável) ou vermelho (nível elevado), criando um alerta visual imediato na interface.

Alerta automático via Telegram:

A quarta saída segue para a função “Se Lux > 400 envia alerta”, com uma lógica semelhante à anterior, mas dedicada ao envio de notificações externas. Se o valor ultrapassar 400 lux, uma mensagem personalizada é formatada e enviada ao nó “Enviar Telegram”, que publica a informação em tempo real para um bot do Telegram

previamente configurado com token e chatId. Isso possibilita que o usuário receba notificações em seu celular ou computador mesmo sem estar com o painel aberto.

Identificação do sensor no painel (ui_text):

Por fim, a quinta saída conecta-se ao nó “Sensor BH1750”, que utiliza o componente ui_text para exibir o nome do sensor diretamente no painel, logo abaixo do título “Lux (Luminosidade)”. Essa identificação contextualiza a origem dos dados, sendo uma boa prática de design em dashboards IoT, principalmente quando há múltiplos sensores operando simultaneamente.

A figura “Dashboard Node-RED com leitura em tempo real da luminosidade” apresenta o painel final do dashboard, disponível em http://localhost:1880/ui, onde os dados de luminosidade são exibidos em tempo real. O título do grupo é “Sensores ESP32”, seguido da variável monitorada “Lux (Luminosidade)”, o nome do sensor em destaque e o valor atual exibido em um gauge circular. Logo abaixo, o alerta de limiar é indicado com texto e LED colorido, que acende automaticamente quando o valor ultrapassa 400 lux.

FIAP

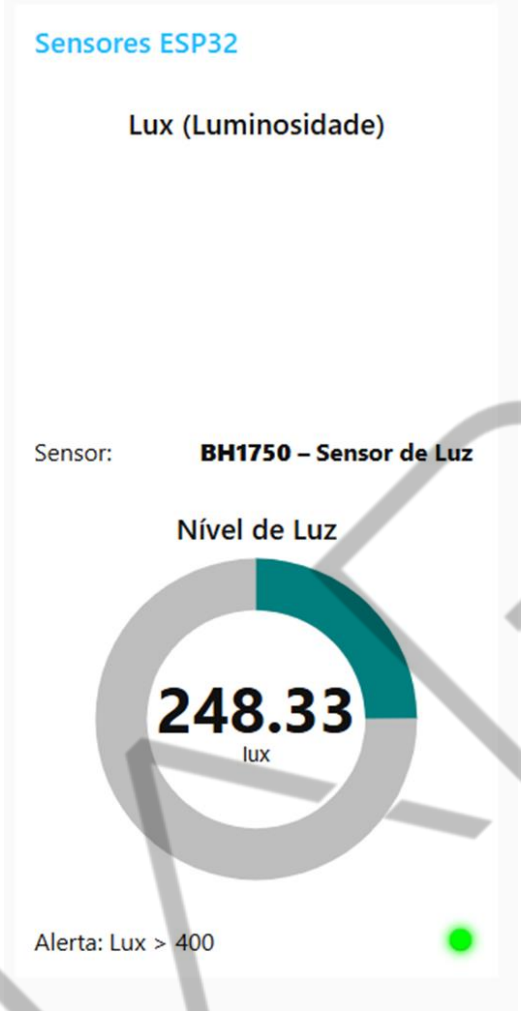


Figura 4 - Dashboard Node-RED com leitura em tempo real da luminosidade
Fonte: Node-RED (2025)

O componente central do painel é o gauge circular, que mostra em tempo real o valor atual da luminosidade em lux, com um formato visual que facilita a leitura rápida, mesmo em ambientes com múltiplas variáveis. O gauge está configurado com uma escala adequada ao sensor, e as cores internas variam conforme o nível de luz. Logo abaixo, a legenda indica a lógica de alerta: “Alerta: Lux > 400”. À direita dessa legenda, um LED virtual está posicionado para indicar visualmente o status atual — aceso em verde quando o valor está abaixo do limiar e em vermelho quando ultrapassado.

Esse ensaio, além de demonstrar a capacidade de monitoramento local e remoto, também mostra como ferramentas visuais como o Node-RED permitem desenvolver interfaces interativas e profissionais mesmo sem conhecimentos avançados de programação. A modularidade do fluxo, a clareza da visualização e a possibilidade de integração com serviços externos como o Telegram tornam esse

modelo altamente replicável para aplicações reais de automação, segurança, agricultura inteligente, controle ambiental, entre outros contextos.

2.4 Criação de tópicos MQTT e leitura em tempo real

Nesta subseção, vamos compreender como funcionam os tópicos MQTT e como realizar a leitura dos dados em tempo real por meio da ferramenta Node-RED. Essa é uma etapa essencial para aplicações em IoT, pois permite a comunicação contínua entre dispositivos e servidores. Aqui, será ilustrada uma aplicação prática utilizando o sensor de luminosidade BH1750 embarcado em um ESP32, com envio dos dados para o broker MQTT e visualização em tempo real.

No protocolo MQTT, as informações são trocadas por meio de tópicos, que funcionam como canais nomeados. Um dispositivo pode publicar informações em um tópico, enquanto outro dispositivo ou sistema pode assinar (subscribe) esse tópico para receber os dados assim que forem enviados.

No nosso exemplo, o ESP32 lê a luminosidade ambiental por meio do sensor BH1750 e, em seguida, publica os dados no tópico esp32/luminosidade. A cada leitura realizada, o valor em lux é empacotado em formato JSON e enviado para o broker.

A seguir, o trecho de código demonstra essa publicação:

```
String payload = "{\"lux\": " + String(lux) + "}";
client.publish("esp32/luminosidade", payload.c_str());
```

Código-fonte 4 - Código de envio de dados do ESP32 para o tópico MQTT
Fonte: Elaborado pelo autor (2025)

No lado do servidor, o Node-RED realiza a assinatura deste mesmo tópico. O nó “MQTT in” é configurado para escutar o tópico esp32/luminosidade em um broker local (por exemplo, localhost:1883). Assim que o dado chega, ele é decodificado de JSON, processado por uma função personalizada e exibido em componentes visuais como gauges e gráficos de linha. Este processo garante a visualização contínua e automática dos dados, sem a necessidade de atualizações manuais.

O fluxograma da figura “Fluxo Node-RED com recepção MQTT em tempo real” mostra esse fluxo de comunicação e visualização, que inclui os seguintes nós:

- **MQTT in:** recebe os dados publicados no tópico;
- **json:** converte o texto recebido em um objeto JSON;

- **function:** separa os valores e define a estrutura de saída;
- **ui_gauge:** exibe o valor atual da luminosidade em lux;
- **ui_chart:** constrói um gráfico de linha para representar a evolução da luminosidade ao longo do tempo.



Figura 5 - Fluxo Node-RED com recepção MQTT em tempo real
Fonte: Node-RED (2025)

Esse fluxo exemplifica como o Node-RED pode ser configurado para capturar e apresentar dados em tempo real, proporcionando um ambiente visual e interativo para aplicações IoT. Ao acessar http://localhost:1880, o usuário pode editar o fluxo no editor gráfico, enquanto em http://localhost:1880/ui, é possível acompanhar a visualização dinâmica no painel de dashboard.

2.5 Leitura de luminosidade com GY-30

Nesta subseção, será apresentada uma aplicação prática de leitura de luminosidade utilizando o sensor GY-30, que incorpora o componente BH1750FVI. Esse sensor é amplamente utilizado em projetos de Internet das Coisas devido à sua precisão, baixo consumo de energia e facilidade de integração. A leitura será realizada por um microcontrolador ESP32, com o envio dos dados por MQTT para visualização em tempo real no Node-RED.

O GY-30 é um módulo digital de luminosidade baseado no sensor BH1750, que permite medir a intensidade da luz ambiente em unidades de lux (lx). Ele se comunica com microcontroladores por meio da interface I^2C, o que facilita a conexão com plataformas como Arduino e ESP32. Possui uma faixa de medição ampla, variando de 1 a 65535 lux, o que o torna ideal para projetos de automação residencial, controle de iluminação, estufas inteligentes, entre outras aplicações.

A conexão do sensor ao ESP32 é realizada através de quatro fios simples. O pino VCC do GY-30 deve ser conectado ao pino de 3.3V do ESP32; o GND do sensor ao GND do microcontrolador; o pino SDA do sensor deve ser ligado ao GPIO 21 do ESP32; e o pino SCL ao GPIO 22. Essa configuração corresponde ao padrão de comunicação I²C da placa, amplamente utilizado em sensores digitais. É importante lembrar que, embora o GY-30 suporte alimentação de 5V, recomenda-se o uso de 3.3V quando utilizado com o ESP32, a fim de proteger os circuitos internos do microcontrolador contra possíveis sobretensões.

A próxima figura ilustra a ligação física entre o sensor GY-30 e o ESP32, destacando claramente o roteamento de cada fio: alimentação (vermelho), terra (preto), linha de dados SDA (verde) e linha de clock SCL (azul). Essa referência visual ajuda na reprodução prática do circuito e evita erros de conexão durante a montagem do projeto.

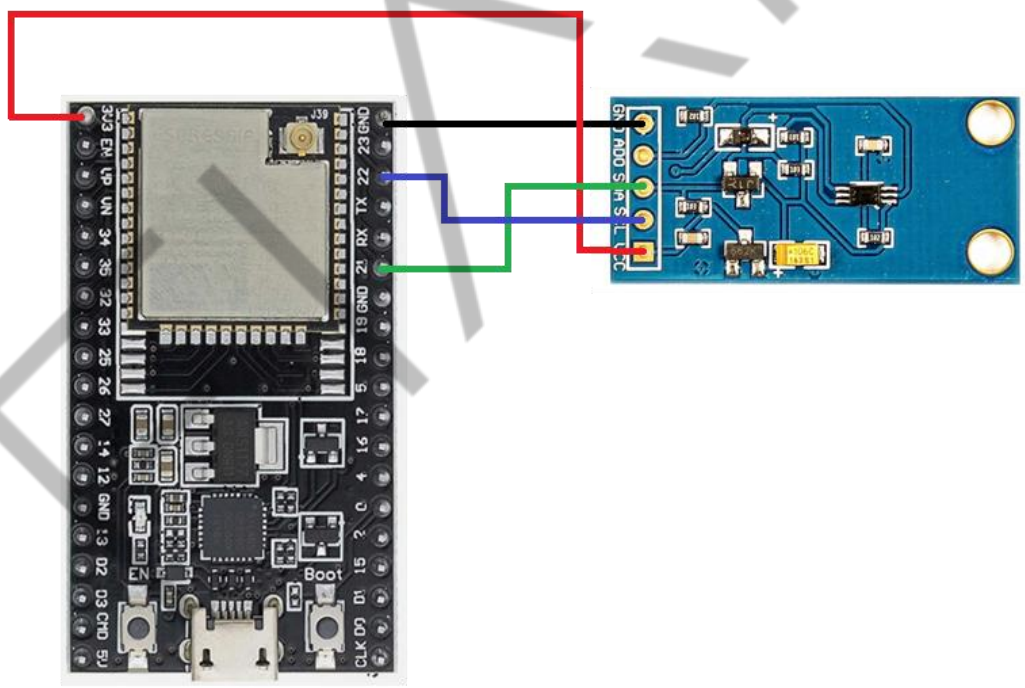


Figura 6 - Conexão física entre o sensor de luminosidade GY-30 (BH1750) e o microcontrolador ESP32 utilizando a interface I²C
Fonte: Elaborado pelo autor (2025)

Após a montagem física, é necessário desenvolver a lógica de funcionamento. O código carregado no ESP32 realiza a leitura da luminosidade ambiente por meio do sensor BH1750, conecta o dispositivo a uma rede Wi-Fi, estabelece comunicação com um broker MQTT (que pode ser local ou em nuvem) e envia os dados formatados no padrão JSON para um tópico específico. A cada leitura, o valor medido em lux é

encapsulado e publicado no tópico esp32/luminosidade, que será assinado pelo Node-RED.

A lógica do programa funciona em ciclos contínuos. Inicialmente, a biblioteca Wire é configurada com os pinos SDA e SCL padrão do ESP32, e a biblioteca do BH1750 inicia o sensor em modo de operação contínua. Em seguida, o ESP32 tenta se conectar ao Wi-Fi com as credenciais fornecidas. Uma vez conectado à internet, ele se comunica com o broker MQTT e se certifica de que está conectado a ele antes de enviar qualquer dado.

Durante o loop principal, o sensor realiza uma leitura do nível de luminosidade, imprime esse valor no Monitor Serial e o publica no tópico MQTT no formato JSON, como por exemplo: {"lux": 248.33}. O código também inclui uma função de reconexão automática com o broker, garantindo que a comunicação seja mantida mesmo em caso de perda de conexão. A cada 2 segundos, uma nova leitura é realizada, mantendo o sistema em operação contínua.

O código a seguir mostra a implementação completa do programa utilizado no ESP32 com o sensor GY-30:

```
    #include <Wire.h>
    #include <BH1750.h>
    #include <WiFi.h>
    #include <PubSubClient.h>

    const char* ssid = "NOME_DA_REDE";
    const char* password = "SENHA_DA_REDE";
    const char* mqtt_server = "broker.emqx.io"; // ou
"192.168.0.X" para broker local

    WiFiClient espClient;
    PubSubClient client(espClient);
    BH1750 lightMeter;

    void setup() {
      Serial.begin(115200);
      Wire.begin(21, 22); // SDA, SCL
      lightMeter.begin();

      WiFi.begin(ssid, password);
      while (WiFi.status() != WL_CONNECTED) {
        delay(1000);
        Serial.println("Conectando ao WiFi...");
      }
```

```
  client.setServer(mqtt_server, 1883);
}


void loop() {
  if (!client.connected()) {
    reconnect();
  }
  client.loop();

  float lux = lightMeter.readLightLevel();
  Serial.print("Luminosidade: ");
  Serial.print(lux);
  Serial.println(" lux");

  String payload = "{\"lux\": " + String(lux, 2) + "}";
  client.publish("esp32/luminosidade", payload.c_str());

  delay(2000);
}

void reconnect() {
  while (!client.connected()) {
    if (client.connect("ESP32Client")) {
      client.subscribe("esp32/luminosidade");
    } else {
      delay(2000);
    }
  }
}
```

Código-fonte 5 - Envio de dados do sensor BH1750 via MQTT utilizando ESP32
Fonte: Elaborado pelo autor (2025)

2.6 Alertas inteligentes com Node-RED

Sistemas de IoT não se limitam apenas à coleta e visualização de dados; eles também desempenham papel fundamental na geração de respostas automáticas e na prevenção de situações críticas. Nesse contexto, os alertas inteligentes representam um recurso essencial, permitindo que ações sejam tomadas de forma imediata com base nos dados capturados pelos sensores e processados por plataformas como o Node-RED.

O Node-RED é particularmente eficaz na construção de sistemas de alerta, pois permite a criação de fluxos lógicos por meio de blocos visuais que podem ser facilmente interligados. Com ele, é possível monitorar continuamente variáveis de sensores e definir condições específicas que, quando satisfeitas, disparam ações

como envio de mensagens para Telegram, e-mails ou chamadas a serviços web. Essa flexibilidade torna o Node-RED uma ferramenta poderosa tanto para prototipação quanto para aplicações práticas em ambientes industriais, residenciais e educacionais.

2.6.1 Condições para disparo de alerta

O primeiro passo para configurar alertas no Node-RED é estabelecer as condições para o disparo. Por exemplo, é possível definir um limite crítico de luminosidade captada por um sensor GY-30 e acionar uma notificação sempre que o valor ultrapassar 300 lux. Essa lógica pode ser implementada com os blocos "switch" e "change", que direcionam o fluxo de dados para diferentes saídas, dependendo das condições estabelecidas. A partir disso, é possível conectar nós de notificação, como o "telegram sender".

2.6.2 Envio de alertas via Telegram

Para permitir que o sistema envie alertas automáticos, é necessário configurar um canal de comunicação. No exemplo a seguir, utiliza-se o Telegram como meio de envio de notificações. O primeiro passo consiste em criar um bot usando o recurso oficial chamado BotFather, por meio do qual se obtém o token de autenticação necessário para habilitar a comunicação com o Node-RED.

No ambiente Node-RED, é necessário instalar o pacote adicional node-red-contrib-telegrambot, que adiciona blocos específicos para envio e recepção de mensagens via Telegram. Após configurar o nó de saída com o token do bot e o chat ID do destinatário, é possível conectá-lo a funções que definem a lógica de quando uma mensagem deve ser enviada.

A seguir, a figura ilustra um exemplo de fluxo implementado para essa finalidade. O nó “Se Lux > 400 envia alerta” contém uma função JavaScript que verifica a transição do nível de luminosidade medido. Essa lógica evita o envio repetitivo de mensagens enquanto o valor permanecer acima de 400 lux, garantindo que o alerta ocorra apenas no momento de transição.

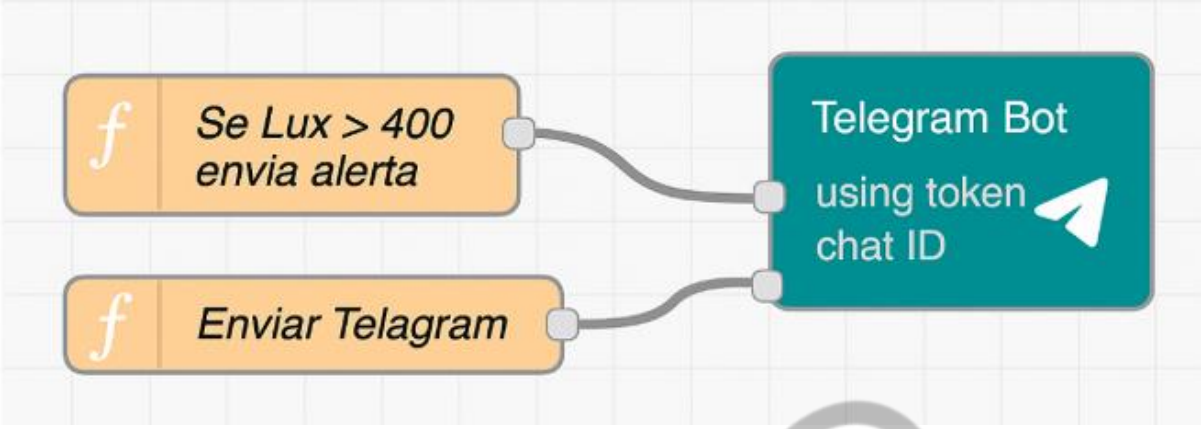


Figura 7 - Fluxo de envio de alerta de luminosidade com integração ao Telegram via Node-RED
Fonte: Node-RED (2025)

A lógica utilizada na função é a seguinte:

```
    let lux = msg.payload.lux;
    let   estadoAnterior   =   context.get("estadoAlerta")   ||
"baixo";

    // Transição de abaixo para acima de 400
    if (lux > 400 && estadoAnterior === "baixo") {
        context.set("estadoAlerta", "alto");
        msg.payload = {
            chatId: 7100155391,  // Substitua pelo seu chatId
se mudar
            type: "message",
            content: `⚠ Alerta: Luminosidade alta detectada:
${lux.toFixed(2)} lux`
        };
        return msg;
    }

    // Transição de acima para abaixo de 400
    if (lux <= 400 && estadoAnterior === "alto") {
        context.set("estadoAlerta", "baixo");
    }

    // Em qualquer outro caso, não envia nada
    return null;
```

Código-fonte 6 - Código de detecção e envio de alerta de luminosidade via Telegram no Node-RED
Fonte: Elaborado pelo autor (2025)

Além do Telegram, o Node-RED também oferece suporte nativo ao envio de e-mails e requisições HTTP. Isso possibilita integração com serviços web como IFTTT, Zapier e APIs personalizadas. Dessa forma, o sistema pode não apenas notificar os usuários, mas também interagir com outros sistemas, registrar eventos em bancos de dados ou acionar dispositivos remotos.

2.6.3 Alerta de luminosidade via Telegram

Na experiência realizada, foi implementado um fluxo de alerta inteligente utilizando o sensor de luminosidade GY-30 (BH1750) conectado a um ESP32. Esse sensor é responsável por medir a intensidade de luz em lux e enviar os dados em tempo real por meio do protocolo MQTT. Os dados são então recebidos pelo Node-RED, que analisa os valores e dispara alertas conforme as condições configuradas.

O fluxo desenvolvido no Node-RED conta com um nó "MQTT in", que escuta o tópico esp32/lux. As mensagens recebidas são convertidas de JSON e, em seguida, avaliadas por um nó do tipo "switch" ou "function", que verifica se o valor ultrapassa 400 lux. Caso a condição seja atendida, um LED virtual vermelho é aceso no painel e uma mensagem é enviada automaticamente via Telegram para o usuário.

A figura “Dashboard com gauge de luminosidade e alerta visual para valores acima de 400 lux” apresenta o painel de controle criado no Node-RED. Nele, é possível visualizar o gauge de luminosidade com leitura em tempo real e o LED indicador de alerta. Quando o valor de lux ultrapassa o limite pré-definido, o sistema responde imediatamente com a mudança de estado do LED e o envio de mensagem ao Telegram. Esse painel demonstra o potencial do Node-RED para monitoramento visual e supervisão remota.

FIAP

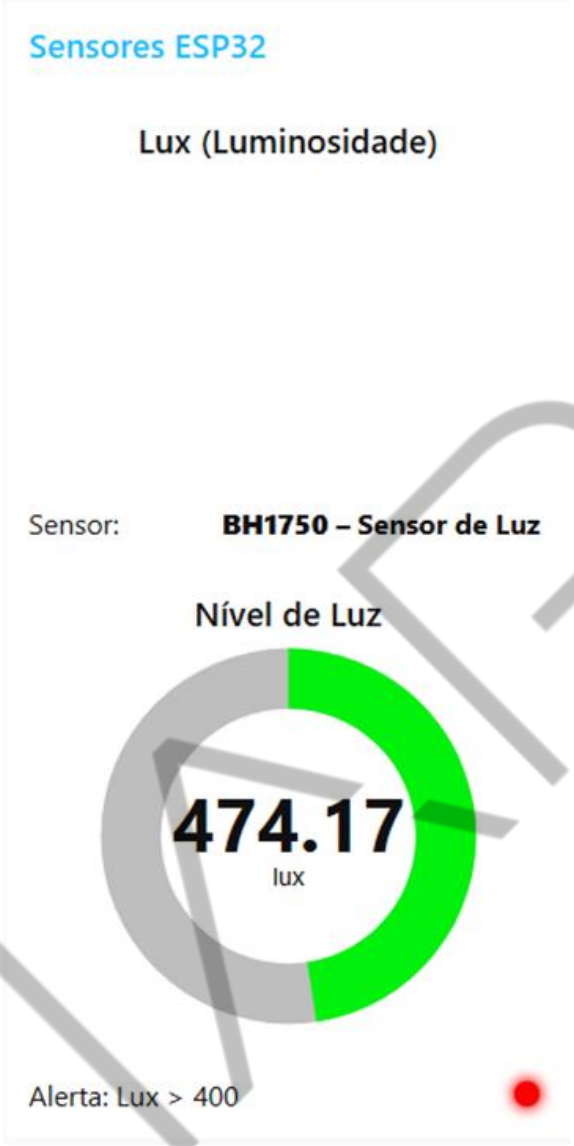


Figura 8 - Dashboard com gauge de luminosidade e alerta visual para valores acima de 400 lux
Fonte: Node-RED (2025)

Após a detecção da condição de risco, o Node-RED utiliza o pacote node-red-contrib-telegrambot, configurado com o token do bot e o chat ID do destinatário, para enviar automaticamente notificações detalhadas. A mensagem informa o valor atual de luminosidade e reforça a ação necessária com um ícone de aviso.

A próxima figura mostra a mensagem de alerta enviada pelo bot do Telegram:

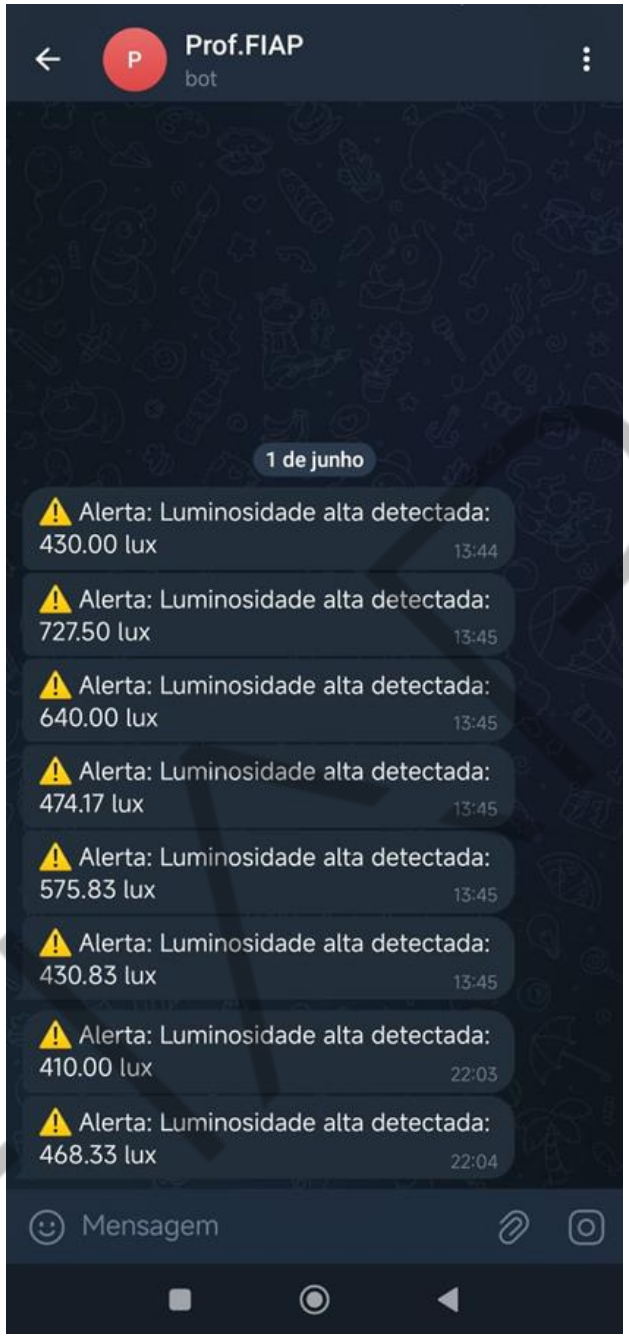


Figura 9 - Alerta enviado via Telegram informando o excesso de luminosidade detectado pelo sensor
Fonte: Elaborado pelo autor (2025)

Essa integração é particularmente útil em ambientes onde o controle de iluminação é crítico, como estufas, laboratórios, escritórios, galpões industriais ou salas de aula. A lógica implementada no fluxo evita o envio repetido de mensagens enquanto o valor permanecer alto, garantindo clareza e evitando sobrecarga de notificações.

Além disso, o painel pode ser expandido com controles deslizantes (*sliders*), que permitem alterar dinamicamente o limiar de alerta — tudo por meio da interface

gráfica, sem a necessidade de alterações no código do microcontrolador. Isso demonstra a flexibilidade e a robustez do Node-RED como plataforma de automação em projetos IoT de baixo custo e alta efetividade.

2.7 Análise dos resultados

A experiência realizada com o sensor GY-30 (BH1750), o microcontrolador ESP32 e a plataforma Node-RED demonstrou, na prática, como é possível construir um sistema de supervisão ciberfísico com resposta em tempo real, utilizando protocolos leves como o MQTT e ferramentas visuais de alto nível. Desde a leitura da luminosidade até a sua representação em tempo real no dashboard, o tempo de resposta observado foi inferior a 100 milissegundos, o que garante fluidez na visualização e confiabilidade na tomada de decisões baseadas nos dados recebidos.

Durante os testes, foi possível observar que o sistema realizava leituras contínuas e estáveis a cada 2 segundos, transmitindo os valores via MQTT, que eram processados pelo Node-RED e imediatamente exibidos em componentes como gauges, gráficos históricos e indicadores visuais de alerta. A simplicidade do protocolo MQTT, aliado à leveza do sensor BH1750 e à eficiência do ESP32, colaboraram para esse desempenho.

Outro ponto positivo foi a flexibilidade do Node-RED, que permitiu construir um painel completo com poucos nós, além de facilitar a adição de novas funcionalidades, como o envio de mensagens via Telegram e a identificação do sensor utilizado. A modularidade da ferramenta é especialmente vantajosa em aplicações IoT, pois permite rápida prototipação, reorganização dos fluxos e fácil manutenção do sistema.

O sistema também se mostrou escalável, já que a lógica implementada pode ser replicada para múltiplos sensores e tópicos diferentes. Basta alterar o nome do tópico MQTT, configurar novos nós de entrada e adaptar os elementos visuais do dashboard. Isso abre espaço para aplicações em redes mais complexas, como ambientes industriais, escolas, fazendas ou prédios automatizados.

Apesar dos bons resultados, algumas limitações foram observadas. O Node-RED é excelente para visualização em tempo real, mas não foi projetado para armazenamento robusto de dados históricos. Além disso, dashboards muito carregados ou com muitos dados simultâneos podem afetar o desempenho em

sistemas embarcados com recursos limitados. A segurança da comunicação MQTT também precisa ser considerada, principalmente em ambientes externos ou críticos, onde o uso de TLS e autenticação com tokens se torna necessário.

De forma geral, a experiência cumpriu plenamente os objetivos didáticos: demonstrar como sensores físicos podem ser integrados a plataformas digitais de supervisão com mínimo esforço de codificação e excelente capacidade de visualização. O uso de tecnologias abertas e gratuitas, como o Node-RED, o MQTT e o ESP32, torna essa solução acessível e replicável em diversos contextos de ensino, pesquisa e inovação tecnológica.

Na próxima seção, será explorado o uso da plataforma Grafana, integrada ao banco de dados InfluxDB, com foco na análise histórica, armazenamento de longo prazo e correlação entre variáveis. Essa abordagem complementa o sistema em tempo real, oferecendo uma visão mais analítica e contextualizada dos dados coletados em projetos IoT.

FIAP

3 GRAFANA + INFLUXDB PARA ANÁLISE HISTÓRICA

A análise de dados históricos é um componente fundamental em sistemas de IoT, especialmente quando se busca identificar tendências, padrões comportamentais ou realizar diagnósticos retrospectivos. Diferentemente das soluções voltadas apenas para dados em tempo real, a visualização histórica exige uma arquitetura que priorize armazenamento eficiente e ferramentas capazes de lidar com grandes volumes de informação ao longo do tempo.

Neste contexto, o uso conjunto do InfluxDB e do Grafana tem se consolidado como uma das abordagens mais eficazes. O InfluxDB, por ser um banco de dados orientado a séries temporais, otimiza a gravação e a recuperação de dados baseados em tempo, como aqueles gerados por sensores ambientais e dispositivos embarcados. Já o Grafana se destaca como uma interface de visualização poderosa, permitindo a construção de dashboards personalizáveis, interativos e com múltiplas fontes de dados.

Esta seção aborda detalhadamente a integração entre essas duas ferramentas no contexto da IoT, a partir de um projeto prático com o ESP32 e o sensor GY-91, demonstrando como armazenar, consultar e visualizar dados com eficiência. A seguir, será apresentado o funcionamento do InfluxDB como banco de séries temporais.

3.1 Introdução ao InfluxDB como banco de séries temporais

InfluxDB é um banco de dados projetado especificamente para armazenar e consultar dados de séries temporais, ou seja, dados que variam ao longo do tempo. Em projetos de IoT, essa característica é essencial, pois sensores embarcados geram leituras contínuas que devem ser registradas com marcação temporal precisa para permitir análises posteriores. Ao contrário de bancos relacionais tradicionais, o InfluxDB otimiza tanto o armazenamento quanto a consulta de dados em função do tempo, com suporte nativo a agregações, seleções por intervalo e análises por janela.

Sua estrutura organiza os dados em "buckets", cada um representando uma fonte de dados distinta, com campos como timestamp, valor e tags (rótulos descritivos). Isso permite consultar apenas as séries de interesse, filtrando por dispositivo, localização ou outro critério.

A figura a seguir apresenta uma representação esquemática da estrutura lógica de armazenamento no InfluxDB.

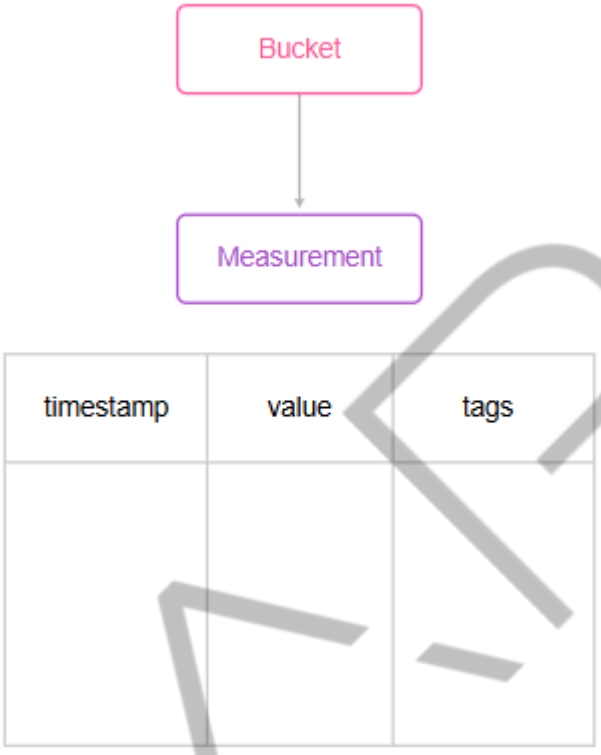


Figura 10 - Estrutura lógica do InfluxDB para dados de sensores IoT
Fonte: Elaborado pelo autor (2025)

3.2 Introdução ao Grafana como ferramenta de visualização

Grafana é uma plataforma de visualização de dados que permite criar dashboards dinâmicos com painéis interativos, muito utilizada em ambientes DevOps, monitoramento industrial e projetos de IoT. A integração com o InfluxDB é nativa, possibilitando montar painéis com várias métricas simultâneas, filtradas por tempo ou agrupadas por categorias.

Com o Grafana, é possível plotar dados em tempo real ou consultar histórico completo, utilizando filtros e comandos personalizáveis. Cada painel pode exibir gráficos de linha, tabelas, indicadores numéricos, mapas de calor e muito mais.

A seguinte figura apresenta uma visualização típica de dados de sensores no Grafana.

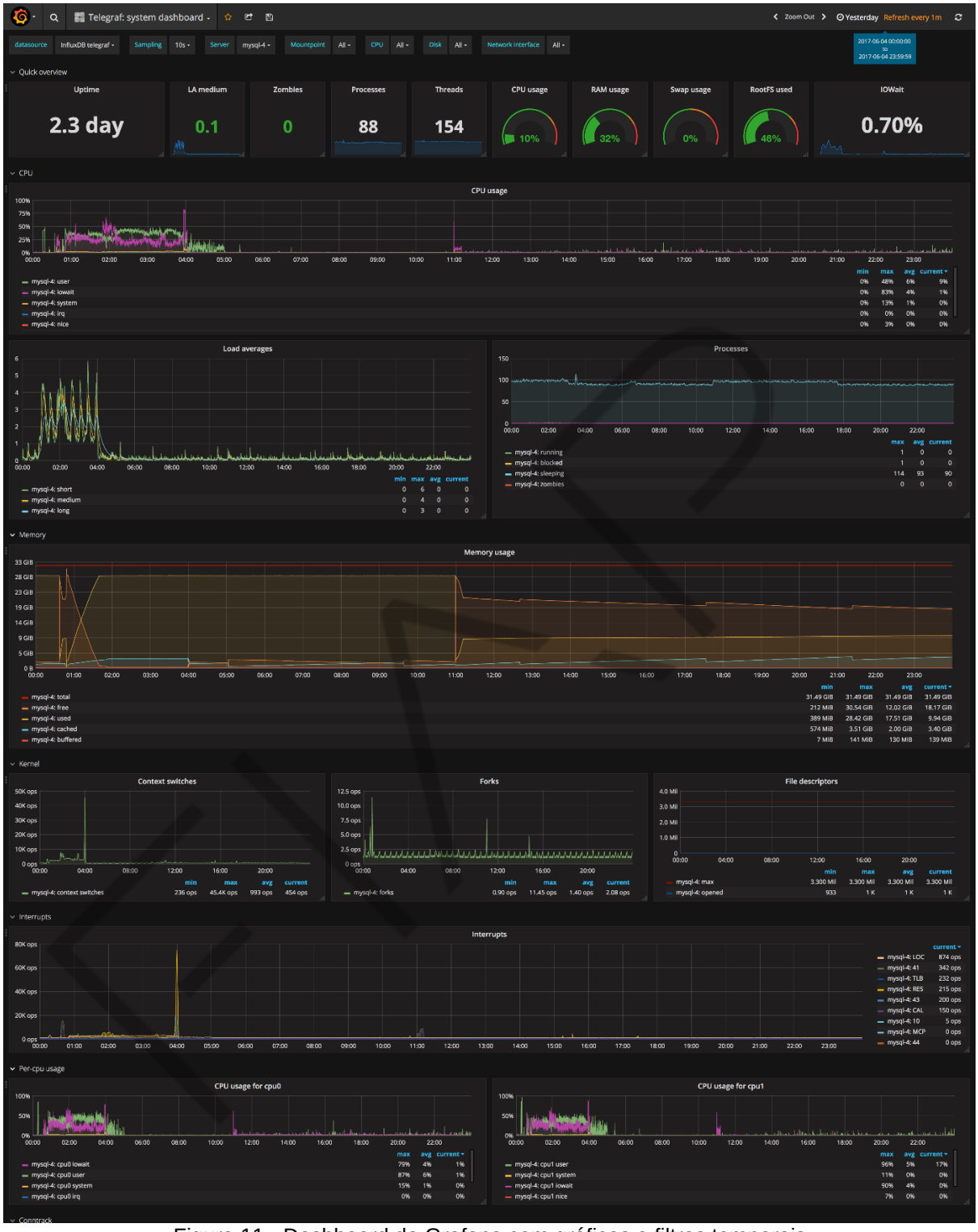


Figura 11 - Dashboard do Grafana com gráficos e filtros temporais
Fonte: Grafana (2025)

3.3 Integração de dados com InfluxDB e visualização em painéis Grafana

A integração de sensores embarcados com bancos de dados de séries temporais é uma etapa fundamental em projetos de IoT. Neste contexto, foi utilizada uma arquitetura composta pelas seguintes tecnologias: ESP32, MQTT, Node-RED,

InfluxDB e Grafana, cada uma cumprindo um papel essencial no ciclo de aquisição, armazenamento e visualização dos dados.

O processo se inicia com o ESP32, microcontrolador responsável por coletar dados de sensores como o GY-91, que mede aceleração, temperatura e pressão atmosférica. Esses dados são processados localmente e enviados por meio do protocolo leve MQTT ao broker Mosquitto, instalado localmente no ambiente de testes.

O Node-RED atua como middleware, recebendo os dados MQTT com nós do tipo mqtt in e encaminhando-os diretamente para o banco de dados InfluxDB por meio do nó influxdb out. Para isso, é necessário configurar corretamente os parâmetros de conexão do InfluxDB no Node-RED, incluindo o endereço do servidor, bucket, organização e token de autenticação. O fluxo construído no Node-RED garante que cada nova leitura do sensor seja automaticamente registrada no InfluxDB com um timestamp, preservando a ordem temporal dos dados.

No lado do microcontrolador, a publicação das leituras é feita em tópicos distintos. Observe o código:

```
     client.publish("sensor/acel_x", String(acelX).c_str());
     client.publish("sensor/temp",
String(temperatura).c_str());
     client.publish("sensor/pressao", String(pressao).c_str());
```

Código-fonte 7 - Publicação de dados do sensor GY-91 em tópicos distintos via MQTT
Fonte: Elaborado pelo autor (2025)

Esses tópicos facilitam a filtragem e o roteamento das mensagens dentro do Node-RED.

A arquitetura geral dessa integração, do sensor até o painel de visualização, pode ser observada na figura “Arquitetura de integração: ESP32, MQTT, InfluxDB, Node-RED e Grafana”. Ela ilustra o caminho completo dos dados: o ESP32 transmite via Wi-Fi para o broker MQTT; em seguida, os dados são então gravados no InfluxDB com auxílio do Node-RED; e por fim, lidos pelo Grafana para exibição.

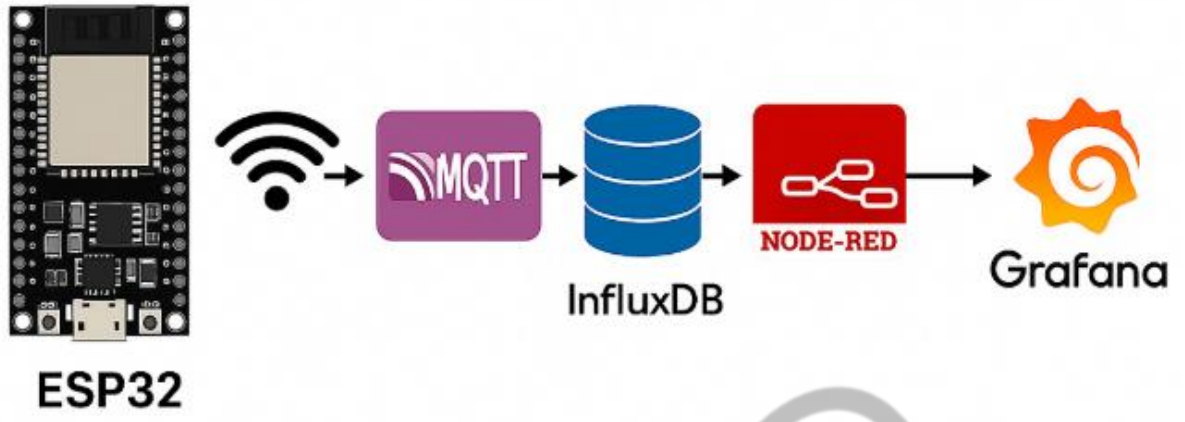


Figura 12 - Arquitetura de integração: ESP32, MQTT, InfluxDB, Node-RED e Grafana
Fonte: Elaborado pelo autor (2025)

Com os dados devidamente armazenados no InfluxDB, a etapa seguinte é a construção de painéis interativos utilizando o Grafana, plataforma especializada na visualização de séries temporais. Inicialmente, é criada uma fonte de dados do tipo InfluxDB, configurando o bucket e o token utilizados previamente no Node-RED. A partir desse ponto, é possível construir painéis personalizados com uma ou mais séries de dados, ajustando filtros de tempo, tipo de agregações, estilo do gráfico e interações dinâmicas com o usuário.

O Grafana permite ainda criar gráficos com múltiplos eixos verticais, ideal para exibir variáveis com unidades e escalas distintas em um único painel. Por exemplo, pode-se representar a aceleração no eixo X em um eixo vertical à esquerda, enquanto a temperatura ambiente é representada em um eixo à direita. Isso possibilita análises comparativas mais eficientes e uma melhor compreensão do comportamento dinâmico dos sensores.

Além disso, os painéis podem ser configurados com filtros de tempo, zoom, alertas visuais e exportação de dados, oferecendo uma interface robusta para engenheiros, técnicos e analistas. A próxima figura apresenta um exemplo real de painel construído no Grafana com dados coletados pelo sensor BH1750, mostrando a variação da luminosidade ao longo de uma janela de tempo com detalhamento técnico na consulta.

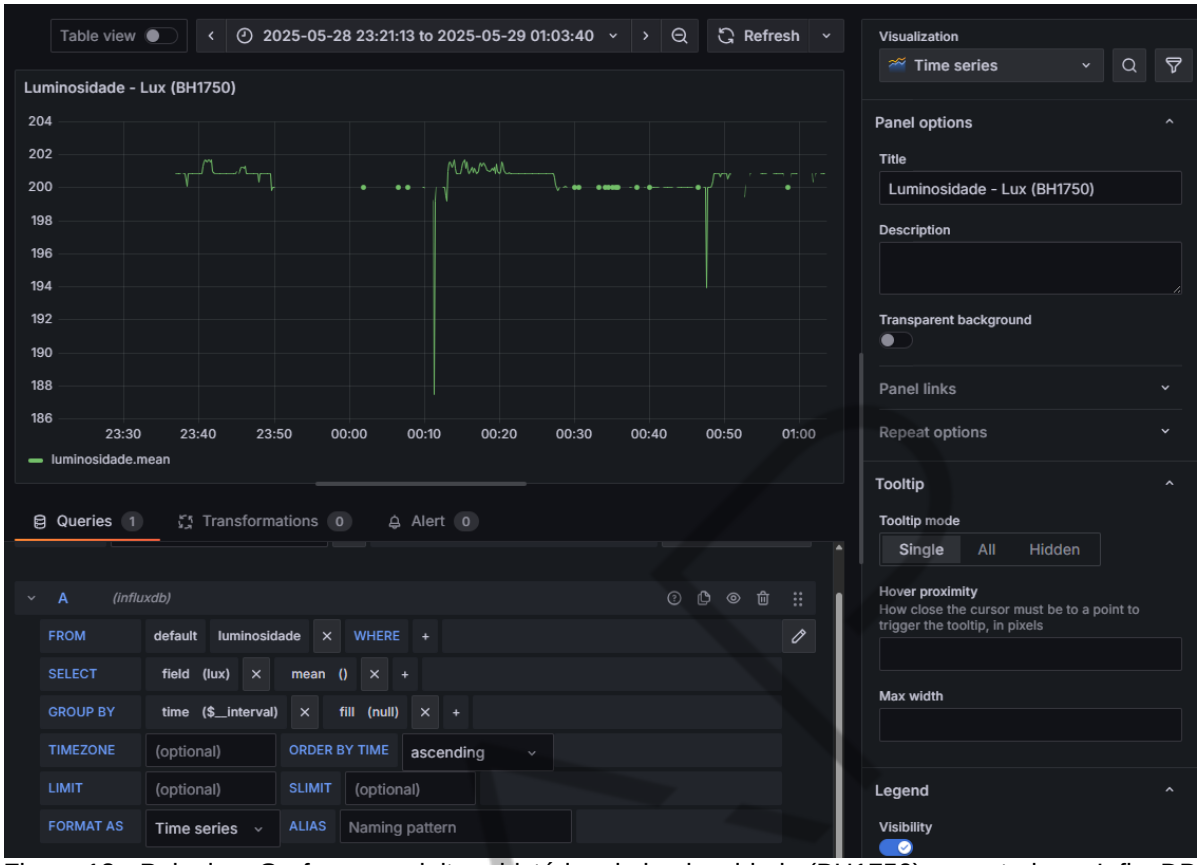


Figura 13 - Painel no Grafana com leitura histórica de luminosidade (BH1750) conectada ao InfluxDB
Fonte: Grafana (2025)

4 THINGSBOARD COMMUNITY EDITION

O ThingsBoard Community Edition é uma plataforma gratuita, de código aberto e baseada em arquitetura modular, desenvolvida para facilitar a implementação de soluções de IoT. Com ele, é possível criar aplicações completas que envolvem a coleta, o armazenamento, a visualização e a análise de dados provenientes de sensores e dispositivos conectados.

A plataforma pode ser executada tanto localmente quanto em servidores na nuvem, e oferece uma interface gráfica responsiva que permite a criação de painéis personalizados com gráficos, indicadores, botões, mapas, tabelas, controles e alarmes. Essa flexibilidade torna o ThingsBoard uma excelente opção para projetos educacionais, experimentais e até aplicações industriais.

A seguir, a figura apresenta um dashboard real construído com a ferramenta, mostrando dados relacionados ao consumo de energia, corrente elétrica, potência, alarmes e outras métricas importantes.

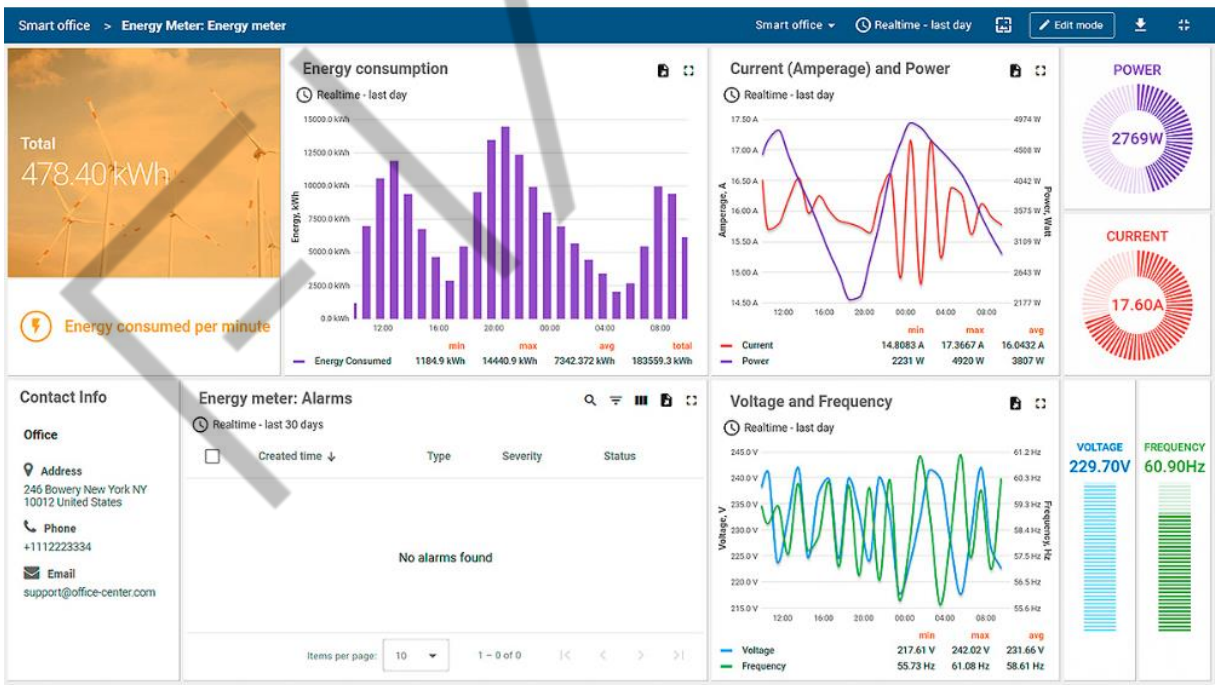


Figura 14 - Exemplo de dashboard no ThingsBoard com indicadores de consumo, corrente, potência, alarmes e contato
Fonte: ThingsBoard (2025)

Cada dispositivo conectado à plataforma é representado por uma entidade digital, que pode enviar dados de telemetria (valores coletados com carimbo de data e hora), armazenar atributos (configurações ou estados persistentes) e receber

comandos (respostas ou ações solicitadas pelo sistema). A comunicação com esses dispositivos é feita, geralmente, por meio de protocolos padronizados como MQTT, HTTP ou CoAP, utilizando mensagens no formato JSON para garantir interoperabilidade e simplicidade de integração.

Esses dispositivos e seus dados são organizados em uma estrutura hierárquica e lógica, onde é possível configurar locações (tenants), clientes, gateways, ativos, alarmes e dashboards específicos. Essa organização facilita tanto o escalonamento da solução quanto a delegação de acessos com diferentes permissões.

A figura a seguir ilustra a arquitetura conceitual do ThingsBoard, detalhando os principais componentes internos da plataforma, como o núcleo central, a engine de regras, o banco de dados SQL e NoSQL, além dos canais de entrada e saída.

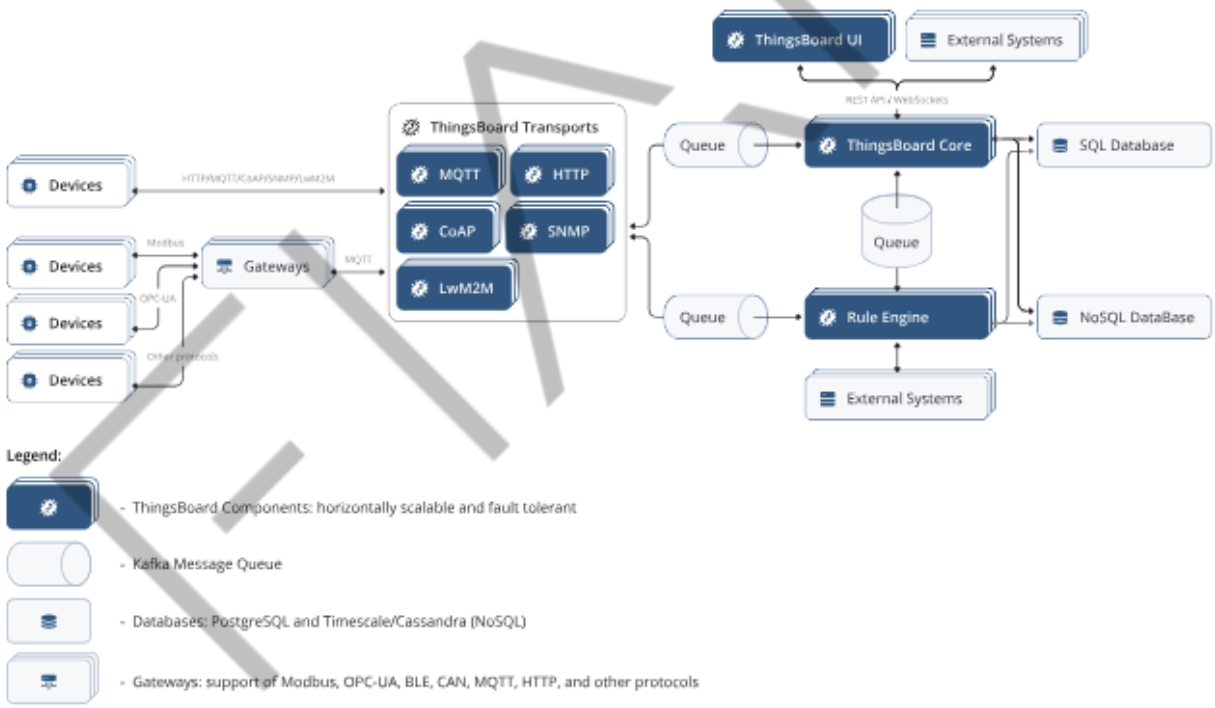


Figura 15 - Arquitetura conceitual do ThingsBoard, destacando fluxos entre dispositivos, protocolos, engine de regras e bases de dados
Fonte: ThingsBoard (2025)

Um dos maiores diferenciais do ThingsBoard em relação a outras plataformas é a presença do Rule Engine, uma ferramenta gráfica que permite definir lógicas de automação para tratamento de eventos recebidos dos dispositivos. Com ele, é possível filtrar dados, aplicar transformações, emitir alertas visuais, acionar comandos externos ou até mesmo registrar os dados em bases externas.

Esses fluxos são criados por meio da conexão de blocos visuais, que representam etapas da lógica, como entrada de dados, verificação de condições, execução de comandos, logs e envio de mensagens. Isso facilita a implementação de regras complexas sem a necessidade de escrever o código manualmente.

A figura “Editor gráfico do Rule Engine, com um fluxo condicional para tratamento de atributos e telemetria” mostra um exemplo prático de uma cadeia de regras criada com o Rule Engine, ilustrando como as mensagens podem ser filtradas, processadas e encaminhadas para diferentes destinos com base em seu conteúdo.

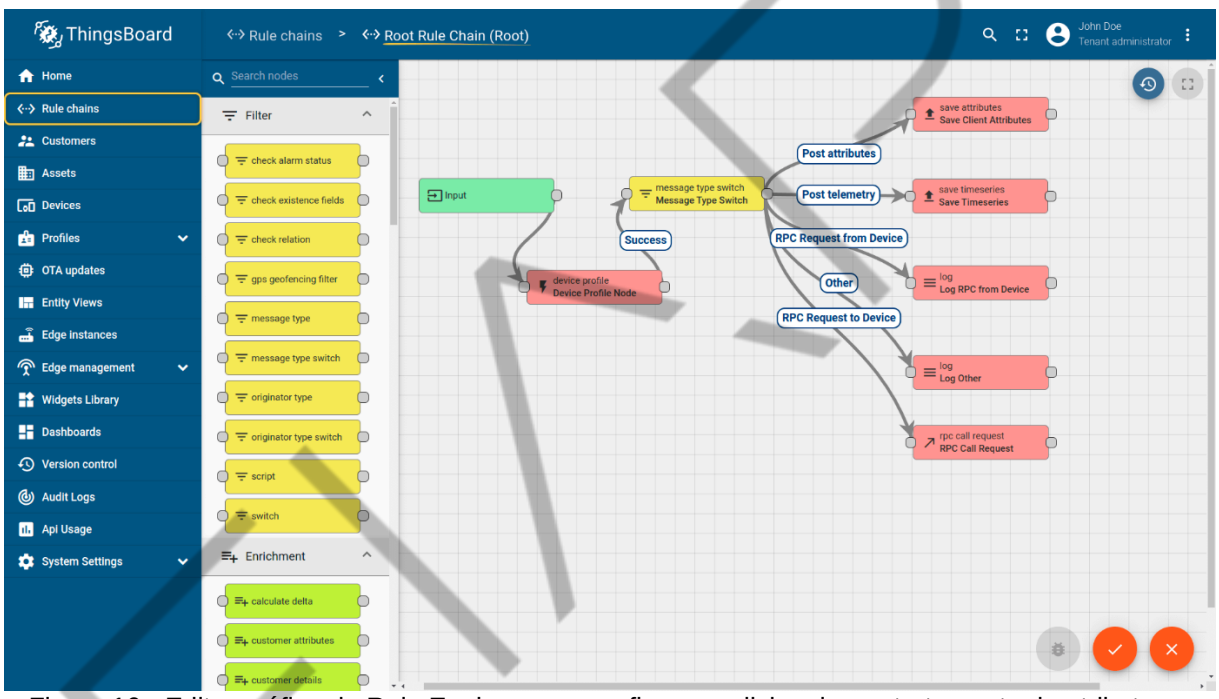


Figura 16 - Editor gráfico do Rule Engine, com um fluxo condicional para tratamento de atributos e telemetria
Fonte: ThingsBoard (2025)

Em resumo, o ThingsBoard Community Edition oferece uma base sólida para desenvolvimento de soluções de IoT que exigem visualização de dados em tempo real, lógica de automação e gestão de múltiplos dispositivos. Mesmo que a experiência prática não tenha sido realizada, o conhecimento da estrutura, funcionamento e potencial de aplicação da plataforma já representa um avanço significativo na compreensão de ferramentas modernas voltadas para o ecossistema IoT.

5 COMPARAÇÃO ENTRE PLATAFORMAS

A escolha da plataforma de visualização adequada é um dos pontos mais importantes no desenvolvimento de sistemas de Internet das Coisas, especialmente quando se deseja garantir robustez, escalabilidade, desempenho eficiente e facilidade de uso. Durante este capítulo, foram utilizadas três plataformas amplamente conhecidas no universo IoT: Node-RED, Grafana e ThingsBoard. A comparação entre essas soluções permite compreender melhor as vantagens e limitações de cada uma, orientando o leitor para decisões mais assertivas na construção de dashboards profissionais ou educacionais.

5.1 Interface visual e curva de aprendizado

A análise comparativa pode ser iniciada pela avaliação da interface visual oferecida por cada ferramenta. O Node-RED apresenta uma interface baseada em fluxos, extremamente intuitiva e de fácil manipulação, o que favorece a aprendizagem de iniciantes e a prototipação rápida de sistemas. Já o Grafana se destaca por oferecer dashboards dinâmicos e com múltiplos tipos de gráficos e painéis interativos. A plataforma ThingsBoard, por sua vez, combina o melhor dos dois mundos, com uma interface rica, interativa e de fácil navegação, pensada para o uso tanto em computadores quanto em dispositivos móveis.

Outro ponto importante está na curva de aprendizado. O Node-RED é reconhecido por sua baixa barreira de entrada, sendo ideal para iniciantes e ambientes educacionais. Em contrapartida, o Grafana exige algum conhecimento prévio em bancos de dados de séries temporais e construção de consultas para geração de gráficos. O ThingsBoard possui uma complexidade maior, principalmente na criação de dashboards vinculados a atributos, dispositivos e regras de automação, exigindo mais tempo e familiaridade com conceitos avançados de IoT.

5.2 Integração, armazenamento e visualização

Em relação à integração com o protocolo MQTT, todas as plataformas oferecem suporte. No Node-RED, essa integração é direta e nativa, com nós específicos para publicação e assinatura. O Grafana depende da intermediação de um banco de dados, como o InfluxDB, que recebe as mensagens MQTT e as

armazena. O ThingsBoard realiza a integração diretamente com o broker, utilizando mensagens em JSON e suporte a autenticação, tornando-se uma solução robusta para ambientes corporativos.

A necessidade ou não de um banco de dados externo também impacta na escolha da plataforma. O Node-RED funciona sem banco de dados, caso o objetivo seja apenas o monitoramento em tempo real. Já o Grafana depende de uma base como InfluxDB, Prometheus ou outros. O ThingsBoard inclui o PostgreSQL como banco interno, facilitando a instalação por meio de containers Docker. Isso garante que os dados possam ser armazenados de forma persistente sem configurações adicionais.

A visualização histórica é outro aspecto central em sistemas IoT. Grafana é a plataforma que mais se destaca nesse aspecto, com múltiplos tipos de gráficos, filtros por período e dashboards altamente personalizáveis. O ThingsBoard também oferece boas opções de visualização histórica e armazenamento, enquanto o Node-RED apresenta limitações nesse quesito, sendo mais indicado para visualização imediata.

5.3 Funcionalidades adicionais, robustez e escalabilidade

No que se refere ao suporte para alertas e notificações, o Node-RED tem como principal ponto forte a integração com serviços externos como Telegram, e-mail e webhooks, sendo altamente flexível e simples de configurar. O ThingsBoard também possui alertas nativos e automação por meio de regras visuais. O Grafana, embora possua plugins para alertas, apresenta suporte mais limitado nesse aspecto.

Quando se trata de controle remoto de dispositivos, o Node-RED permite essa funcionalidade, mas geralmente exige customizações. O ThingsBoard é o que oferece melhor suporte, com atributos de controle configuráveis diretamente na interface do dashboard, sendo possível acionar dispositivos à distância de maneira intuitiva. O Grafana, por sua vez, não é voltado para esse tipo de funcionalidade.

Ao considerar o quesito da robustez, o ThingsBoard apresenta uma estrutura sólida baseada em microserviços, adequada para implantações em ambientes empresariais. O Grafana é extremamente confiável para análise de dados, enquanto o Node-RED, embora estável, é mais sensível a erros causados por fluxos mal configurados. Em termos de desempenho, o Grafana se sobressai com seu motor

otimizado para grandes volumes de dados, enquanto o ThingsBoard apresenta performance equilibrada. O Node-RED, apesar de mais simples, oferece ótimo desempenho em projetos de pequena e média escala.

Em relação à escalabilidade, o ThingsBoard é o mais indicado, pois oferece suporte a múltiplos dispositivos, autenticação refinada, gestão de tenants e implantação em nuvem. O Grafana possui recursos escaláveis por meio de plugins e integrações, e o Node-RED pode ser escalado com esforço adicional, mas não foi projetado originalmente para ambientes de alta complexidade.

5.4 Quadro comparativo

A comparação entre as plataformas é sintetizada no quadro a seguir, detalhando critérios como integração, visualização, suporte a alertas e banco de dados, além de outros aspectos relevantes. Esse quadro pode servir como referência rápida para tomada de decisão em diferentes projetos.

Critério	Node-RED	Grafana	ThingsBoard
Interface visual	Simples, baseada em fluxos	Avançada, dashboards dinâmicos	Completa e interativa
Curva de aprendizado	Baixa	Média	Média a alta
Integração com MQTT	Direta e nativa	Via InfluxDB ou outros	Nativa via JSON MQTT
Suporte a banco de dados	Opcional	Requer InfluxDB, Prometheus etc.	Interno (PostgreSQL)
Visualização histórica	Limitada	Muito robusta	Robusta com controle de dados
Alertas	Possui (Telegram, e-mail, HTTP)	Limitado (plugins)	Nativo
Controle remoto	Possível com customização	Não suportado diretamente	Nativo
Open Source	Sim	Sim	Sim
Comunidade e suporte	Ampla	Muito ampla	Em crescimento

Quadro 2 - Comparativo entre plataformas de dashboard para IoT
Fonte: Elaborado pelo autor (2025)

REFERÊNCIAS

O'LEARY, Nick; CONWAY-JONES, Dave. Low-code programming for event-driven applications. **Node-RED**, 04 de novembro de 2024. Disponível em: <https://nodered.org/>. Acesso em: 27 jun. 2025.

OPEN-SOURCE IoT platform. **Things Board**, 2025. Disponível em: <https://thingsboard.io/>. Acesso em: 27 jun. 2025.

THE open observability platform. **Grafana**, 2025. Disponível em: <https://grafana.com/>. Acesso em: 27 jun. 2025.

FIAP